

A Report on the Implementation and Performance of a Image Processing Algorithm Using MPI

Exam No. B024703

November 30, 2016

1 Introduction

This report details the implementation and performance of a parallel message passing code. This code performs an iterative inverse edge detection algorithm using a 2d Von Neumann stencil. For this algorithm the result of each output pixel depends on that of its neighbours in the input image. This made parallelising the code non-trivial, as it required boundary swaps to occur before each iteration. This was further complicated in this case by the selection of a two dimensional domain decomposition.

This report presents both a detailed description of the algorithm and its implementation, then shows the performance achieved with the resulting code before concluding.

2 Methods

2.1 The Algorithm

The code is designed to perform the inverse of an edge detection algorithm. In that algorithm, each element, $edge_{i,j}$, is replaced as follows:

$$edge_{i,j} = image_{i-1,j} + image_{i+1,j} + image_{i,j-1} + image_{i,j+1} - 4image_{i,j}$$

To invert this algorithm and reconstruct the original image an iterative process must be used. In this process, for each iteration, a new two dimensional array is generated as follows:

$$new_{i,j} = 1/4(old_{i1,j} + old_{i+1,j} + old_{i,j1} + old_{i,j+1} - edge_{i,j})$$

After this the old and new arrays are swapped in memory and the process is repeated.

2.1.1 Boundary Conditions

The resulting value of $new_{i,j}$ in each case depends on its surrounding values in the old array. This presents an interesting problem as to what to do in the edge cases, where surrounding values do not completely exist.

In the vertical case, where the calculation required values that were above or below the existing old array, cyclical boundary conditions were used. This means that when the calculation required values one line below the old image, it instead used those from the top line of the old image and vice versa.

In the horizontal case, "sawtooth" boundary conditions were used, where values forming a smooth gradient ranging between 0 and 255 were used on both sides of the image. Detailed discussion of this boundary condition goes beyond the scope of this report.

2.1.2 Parallelisation

The code was parallelised using a two dimensional domain decomposition. This means that the initial 2d array containing our edge image was split into N_x sections in the x direction and N_y sections in the y direction, where $N_x \cdot N_y = P$, and P is the total number of processes.

For the same reasons as discussed in 2.1.1 the case of what to do at the edge of the image provides problems in parallelisation. To find the values required that are outside the range of each process's local array boundary swaps must occur. The neighboring boundaries to each process's local image are appended to that image to form a halo. This boundary swap was repeated before each iteration. Full details of how this was implemented are discussed in 2.2.

This means that once these halos are included, each process must perform the iterative process on an array of size $(N_x + 2) \cdot (N_y + 2)$.

2.1.3 Stopping Condition

For testing performance and accuracy compared to a serial version, it was useful to be able to terminate the algorithm after a fixed number of iterations. However, it is possible to calculate, according to a certain precision, when the iterative process is complete. This is done on the basis that once the algorithm has reached a perfect solution, if the algorithm is stable, then there will be no further changes between iterations. While this perfection may never be achievable, we can say that if the output changes below a certain amount then we can declare our image to be suitably similar to our original and terminate our iterative process.

In terms of calculation, this is done by calculating a value of $\Delta_{i,j}$ for every element in the two dimensional array:

$$\Delta_{i,j} = \max(new_{i,j} - old_{i,j})$$

and comparing the largest of these values with a pre-determined value.

This stopping condition process is relatively straightforward to parallelise. Each process can calculate its own $\max(\Delta_{i,j})$ and these individual values can be gathered and compared to find the maximum of these maximums by a master thread. Thinking in terms of Amdahl's law, it could be said that almost all of this stopping condition calculation is parallelisable, apart from a comparison of P floating point values, where P is the total number of processes.

Nevertheless, this process is relatively computationally expensive. To prevent this process having too strong a negative impact on performance, it was decided that it would only be completed every 20th iteration.

2.2 Implementation

The algorithm described above was implemented in parallel using MPI.

2.3 Compiling and Running the Original Code

To check the correctness of any subsequent version, the original code was compiled in its original form. This form could be thought of as an explicitly implemented version of the static loop scheduling option in OpenMP, with the iterations of the loops evenly divided between threads. This compilation was done using the Intel C Compiler with the `-O3` option specified at compile time for maximum serial optimisation. Beyond using this to verify the correctness of any future solution and that performance was credible, analysis of the performance of this code falls outside the scope of this report.

3 Results

3.1 Correctness

For almost all HPC applications correctness is the primary concern. The code provided included a check sum that could be used to verify the correctness of the affinity results. In all cases this sum yielded the same result as the code provided. We can therefore disregard correctness as a factor when comparing the different schedules.

3.2 Comparative Execution Times of Schedules

The first results gathered were execution times for each loop with both the affinity scheduling and the best loop schedule that comes by default with any OpenMP implementation. These best schedules were determined and discussed in an earlier report. The best schedule for loop 1 was `dynamic, 32` while for loop 2 the results for `dynamic, 8` and `dynamic, 16` were similarly optimal enough that they are both used here. The results for the execution time of loops 1 and 2 are shown in figures ?? and ?? respectively.

3.2.1 Loop 1

For loop 1 we see that performance yielded from the loops running with affinity scheduling is almost identical to that yielded from the `dynamic, 32` schedule.

Such similar performance is explainable, as the scheduling techniques are quite similar in theory. As explained in [1], the **dynamic, 32** schedule has one work queue that is shared between all threads. Each thread claims a contiguous block of 32 iterations from this queue and executes it before going back to the queue to claim another. As the loop in question is natural fairly well load balanced, or each iteration takes a similar amount of time to execute, the main differences between performance from different schedules on this loop come from the overhead of assigning work. Although this work assignation process happens very differently on the two schedules trialled on this loop, the end result seems to be similar. It is worth noting that the affinity schedule does not gain significant performance improvements from cache coherency in this case.

It is likely the same issues become limiting factors for horizontal scalability once the number of processes reaches about 12. This is that threads have to wait to be assigned work for reasons of synchronisation for both these strategies.

3.2.2 Loop 2

Loop 2 shows some more interesting differences between scalability of the two strategies. The **dynamic, 8** and **dynamic, 16** perform similarly to each other until the number of threads reaches 4. At this point no further speed up is shown where the block size is 16. This is because this loop is inherently badly load balanced, meaning that the slowest block of size 16 becomes the dominating factor of the computation time when the **dynamic, 16** schedule is selected. Similarly, the **dynamic, 8** schedule's performance is limited by the time taken for the slowest block of 8 to execute. As expected this approximately half the time taken for the slowest 16 iterations to execute, meaning that the shortest time achieved using this schedule is approximately half that of the time achieved with the larger block size.

For the affinity scheduling the initial block size can be thought of as proportional to $1/p^2$, where p is the total number of threads. This is because each thread starts off with a local set of $s = 1/p$ iterations, before iterating through a block of s/p . This explains the affinity schedule's relatively poor performance where p is smaller, as this corresponds to a large effective block size. The total number of iterations executed was $T = 729$. Therefore, if the execution time of the slowest block were a completely dominating factor, we would expect the performance of the affinity schedule to eclipse that of **dynamic, 16** where $T/p^2 = 16$, or $p = 6.75$ and that of **dynamic, 8** where $T/p^2 = 16$, or $p = 9.5$. Interpolating from figure ?? we can see these calculated figures are broadly correct. The fact that it is more like

$p = 11$ before the performance of affinity and `dynamic`, 16 could be explained by the greater deal of synchronisation required for the affinity schedule. This is because the blocks become smaller as the process goes on, meaning more time is spent fetching work - requiring threads to enter or wait to enter a critical region.

4 conclusion

In conclusion we see that affinity scheduling is a broadly effective strategy. For at least some values of p this schedule matched or bettered the performance of the best option from the OpenMP standard. This is no small feat considering the OpenMP implementation used has the full weight of the mighty Intel Corporation behind it, and the schedules themselves may or may not have been implemented using a lower level, faster framework than OpenMP itself. This suggests that the implementation of this schedule by the author was of high quality.

However, there were no demonstrable speed ups shown to be from cache coherency effects, which is one of the justifications for this technique. Indeed, it seems likely that in the loop 2 case that affinity scheduling only provided higher horizontal scalability because of a lower effective block size. It is therefore likely that similar or better performance could be achieved where p is larger simply by using the dynamic schedule with a smaller block size. This would be a worthwhile future experiment, but goes beyond the scope of this report.

References

- [1] Intel Corp. *OpenMP Loop Scheduling* <https://software.intel.com/en-us/articles/openmp-loop-scheduling> Accessed: 26-10-2016

A Full Results

Table 1: Loop 1

Schedule	No. Threads	Mean Time (s)	Std. Deviation (s)
dynamic,32	1	1.64	0.00
dynamic,32	2	0.83	0.00
dynamic,32	4	0.43	0.01
dynamic,32	6	0.30	0.01
dynamic,32	8	0.23	0.01
dynamic,32	12	0.16	0.01
dynamic,32	16	0.13	0.01
affinity	1	1.65	0.00
affinity	2	0.83	0.00
affinity	4	0.46	0.00
affinity	6	0.32	0.03
affinity	8	0.24	0.01
affinity	12	0.19	0.01
affinity	16	0.18	0.01

Table 2: Loop 2

Schedule	No. Threads	Mean Time (s)	Std. Deviation (s)
dynamic,8	1	8.84	0.01
dynamic,8	2	4.47	0.02
dynamic,8	4	2.24	0.01
dynamic,8	6	1.57	0.00
dynamic,8	8	1.18	0.00
dynamic,8	12	1.07	0.01
dynamic,8	16	1.12	0.10
dynamic,16	1	8.84	0.00
dynamic,16	2	4.47	0.02
dynamic,16	4	2.23	0.01
dynamic,16	6	2.10	0.01
dynamic,16	8	2.10	0.01
dynamic,16	12	2.15	0.09
dynamic,16	16	2.12	0.04
affinity	1	1.65	0.00
affinity	2	0.83	0.00
affinity	4	0.46	0.01
affinity	6	0.32	0.03
affinity	8	0.24	0.01
affinity	12	0.19	0.01
affinity	16	0.18	0.01

B Code Snippet

```
int get_work(int *remaining, int *end_index, int myid, int nthreads, int *lo,
            int *hi)
{
    int i, chunk_size, steal_index, temp_max;
    temp_max = 1;
    // if a thread has no more work it should steal work.
    if (remaining[myid] == 0)
    {
        // use loop to find thread with most work remaining
        temp_max = 0;
        int i;
        for (i = 0; i < nthreads - 1; i++)
        {
            if (temp_max < remaining[i])
            {
                temp_max = remaining[i];
                steal_index = i;
            }
        }
    }
    else
    {
        steal_index = myid;
    }

    if (temp_max != 0)
    {
        chunk_size = (int)ceil((double)remaining[steal_index] / (double)nthreads);
        *lo = end_index[steal_index] - remaining[steal_index];
        *hi = *lo + chunk_size;
        remaining[steal_index] -= chunk_size;
        return 0;
    }
    else
    {
        return 1;
    }
}
```